

Test Report

1. Test Results

The system was tested using **pytest**. Unit, integration, persistence, error-handling, and end-to-end tests were performed.

1.1 test_models.py

Purpose:

Tests the Patient, Doctor, and Appointment models.

Result: PASS

```
(.venv) PS C:\Users\KEERTHANA\OneDrive\Desktop\MCW Training\Hospital_Management_System\smart-hospital-management> pytest tests/test_models.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\KEERTHANA\OneDrive\Desktop\MCW Training\Hospital_Management_System\smart-hospital-management\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\KEERTHANA\OneDrive\Desktop\MCW Training\Hospital_Management_System\smart-hospital-management
collected 5 items

tests/test_models.py::test_patient_creation PASSED [ 20%]
tests/test_models.py::test_doctor_creation PASSED [ 40%]
tests/test_models.py::test_appointment_creation PASSED [ 60%]
tests/test_models.py::test_doctors_have_independent_slot_lists PASSED [ 80%]
]
tests/test_models.py::test_patient_medical_history_defaults_to_none PASSED [100%]

===== 5 passed in 0.03s =====
```

1.2 test_exceptions.py

Purpose:

Tests the custom exception hierarchy.

Result: PASS

```
tests/test_exceptions.py::test_patient_not_found_error PASSED [ 16%]
tests/test_exceptions.py::test_doctor_not_found_error PASSED [ 33%]
tests/test_exceptions.py::test_slot_unavailable_error PASSED [ 50%]
tests/test_exceptions.py::test_invalid_appointment_error PASSED [ 66%]
tests/test_exceptions.py::test_invalid_billing_error PASSED [ 83%]
tests/test_exceptions.py::test_all_errors_inherit_from_hospital_error PASSED [100%]

===== 6 passed in 0.04s =====
```

1.3 test_emergency.py

Purpose:

Tests the emergency priority queue and priority handling.

Result: PASS

```
tests/test_exceptions.py::test_patient_not_found_error PASSED [ 16%]
tests/test_exceptions.py::test_doctor_not_found_error PASSED [ 33%]
tests/test_exceptions.py::test_slot_unavailable_error PASSED [ 50%]
tests/test_exceptions.py::test_invalid_appointment_error PASSED [ 66%]
tests/test_exceptions.py::test_invalid_billing_error PASSED [ 83%]
tests/test_exceptions.py::test_all_errors_inherit_from_hospital_error PASSED [100%]
D [100%]

===== 6 passed in 0.04s =====
```

1.4 test_medical_record.py

Purpose:

Tests medical history storage, retrieval, and filtering.

Result: PASS

```
tests/test_medical_record.py::test_append_visit PASSED [ 10%]
tests/test_medical_record.py::test_multiple_visits_are_preserved PASSED [ 20%]
tests/test_medical_record.py::test_get_full_history PASSED [ 30%]
tests/test_medical_record.py::test_filter_history_by_doctor PASSED [ 40%]
tests/test_medical_record.py::test_empty_history PASSED [ 50%]
tests/test_medical_record.py::test_visit_is_immutable PASSED [ 60%]
tests/test_medical_record.py::test_stream_history_returns_generator PASSED [ 70%]
tests/test_medical_record.py::test_stream_history_iteration PASSED [ 80%]
tests/test_medical_record.py::test_stream_history_empty PASSED [ 90%]
tests/test_medical_record.py::test_filtered_stream_history PASSED [100%]

===== 10 passed in 0.04s =====
```

1.5 test_medical_record_file.py

Purpose:

Tests safe medical-record file handling using the context manager.

Result: PASS

```

tests/test_medical_record_file.py::test_successful_write PASSED [ 20%]
tests/test_medical_record_file.py::test_successful_write_replaces_old_file
PASSED [ 40%]
tests/test_medical_record_file.py::test_failed_write_preserves_original_file
PASSED [ 60%]
tests/test_medical_record_file.py::test_failed_write_does_not_leave_temporary_file
PASSED [ 80%]
tests/test_medical_record_file.py::test_empty_records_create_empty_file PAS
SED [100%]

===== 5 passed in 0.17s =====

```

1.6 test_billing.py

Purpose:

Tests standard billing, insurance billing, and discount calculation.

Result: PASS

```

tests/test_billing.py::test_standard_billing PASSED [ 11%]
tests/test_billing.py::test_standard_itemized_bill PASSED [ 22%]
tests/test_billing.py::test_standard_summary PASSED [ 33%]
tests/test_billing.py::test_insurance_billing PASSED [ 44%]
tests/test_billing.py::test_full_insurance_discount PASSED [ 55%]
tests/test_billing.py::test_invalid_negative_consultation PASSED [ 66%]
tests/test_billing.py::test_invalid_test_charge PASSED [ 77%]
tests/test_billing.py::test_invalid_procedure_charge PASSED [ 88%]
tests/test_billing.py::test_invalid_insurance_rate PASSED [100%]

===== 9 passed in 0.04s =====

```

1.7 test_persistence.py

Purpose:

Tests saving and loading hospital data using JSON.

Result: PASS

```

tests/test_persistence.py::test_save_and_load_patients PASSED [ 14%]
tests/test_persistence.py::test_save_and_load_doctors PASSED [ 28%]
tests/test_persistence.py::test_save_and_load_appointments PASSED [ 42%]
tests/test_persistence.py::test_save_and_load_medical_history PASSED [ 57%]
tests/test_persistence.py::test_missing_files_return_empty_data PASSED [ 71%]
tests/test_persistence.py::test_malformed_patient_json PASSED [ 85%]
tests/test_persistence.py::test_state_survives_restart PASSED [100%]

===== 7 passed in 0.14s =====

```

1.8 test_logger.py

Purpose:

Tests application logging functionality.

Result: PASS

```
tests/test_logger.py::test_logger_creates_log_file PASSED [ 20%]
tests/test_logger.py::test_patient_registration_is_logged PASSED [ 40%]
tests/test_logger.py::test_appointment_events_are_logged PASSED [ 60%]
tests/test_logger.py::test_emergency_and_billing_are_logged PASSED [ 80%]
tests/test_logger.py::test_error_is_logged PASSED [100%]

===== 5 passed in 0.11s =====
```

1.9 test_decorators.py

Purpose:

Tests the @audit_log decorator.

Result: PASS

```
tests/test_decorators.py::test_audit_log_records_operation PASSED [ 20%]
tests/test_decorators.py::test_audit_records_user PASSED [ 40%]
tests/test_decorators.py::test_audit_records_operation PASSED [ 60%]
tests/test_decorators.py::test_audit_returns_function_result PASSED [ 80%]
tests/test_decorators.py::test_audit_entry_is_created PASSED [100%]

===== 5 passed in 0.10s =====
```

1.10 test_audit_integration.py

Purpose:

Tests audit logging integration with hospital operations.

Result: PASS

```
tests/test_audit_integration.py::test_booking_creates_audit_log PASSED [ 33%]
tests/test_audit_integration.py::test_emergency_creates_audit_log PASSED [ 66%]
tests/test_audit_integration.py::test_billing_creates_audit_log PASSED [100%]

===== 3 passed in 0.11s =====
```

1.11 test_notifications.py

Purpose:

Tests appointment confirmation, reminders, and notification processing.

Result: PASS

```
tests/test_notifications.py::test_confirmation PASSED [ 20%]
tests/test_notifications.py::test_reminder PASSED [ 40%]
tests/test_notifications.py::test_notification_order PASSED [ 60%]
tests/test_notifications.py::test_empty_queue PASSED [ 80%]
tests/test_notifications.py::test_notification_logging PASSED [100%]

===== 5 passed in 0.07s =====
```

1.12 test_appointment.py

Purpose:

Tests booking, rescheduling, cancellation, completion, and slot conflicts.

Result: PASS

```
tests/test_appointment.py::test_book_appointment PASSED [ 12%]
tests/test_appointment.py::test_invalid_patient PASSED [ 25%]
tests/test_appointment.py::test_double_booking PASSED [ 37%]
tests/test_appointment.py::test_reschedule_appointment PASSED [ 50%]
tests/test_appointment.py::test_cancel_appointment PASSED [ 62%]
tests/test_appointment.py::test_complete_appointment PASSED [ 75%]
tests/test_appointment.py::test_cancelled_appointment_cannot_be_completed PASSED [ 87%]
tests/test_appointment.py::test_complete_missing_appointment_raises_error PASSED [100%]

===== 8 passed in 0.14s =====
```

1.13 test_service.py

Purpose:

Tests the main HospitalService operations.

Result: PASS

```
tests/test_service.py::test_register_patient PASSED [ 20%]
tests/test_service.py::test_book_appointment PASSED [ 40%]
tests/test_service.py::test_admit_emergency PASSED [ 60%]
tests/test_service.py::test_generate_bill PASSED [ 80%]
tests/test_service.py::test_get_reports PASSED [100%]

===== 5 passed in 0.11s =====
```

1.14 test_reports.py

Purpose:

Tests patient count, doctor utilization, revenue, and waiting-time reports.

Result: PASS

```

tests/test_reports.py::test_doctor_utilization PASSED [ 16%]
tests/test_reports.py::test_revenue_collected PASSED [ 33%]
tests/test_reports.py::test_average_emergency_wait_time PASSED [ 50%]
]
tests/test_reports.py::test_all_reports PASSED [ 66%]
tests/test_reports.py::test_patients_seen_per_day_with_no_completed_appointments PASSED [ 83%]
tests/test_reports.py::test_completed_appointment_is_counted_as_patient_seen PASSED [100%]

===== 6 passed in 0.07s =====

```

1.15 test_csv_reports.py

Purpose:

Tests daily report generation in CSV format.

Result: PASS

```

tests/test_csv_reports.py::test_daily_report_csv_is_created PASSED [ 25%]
tests/test_csv_reports.py::test_daily_report_csv_contains_reports PASSED [ 50%]
tests/test_csv_reports.py::test_daily_report_csv_contains_revenue PASSED [ 75%]
tests/test_csv_reports.py::test_daily_report_csv_can_be_read PASSED [100%]

===== 4 passed in 0.11s =====

```

1.16 test_restart_persistence.py

Purpose:

Tests whether data remains available after restarting the application.

Result: PASS

```

tests/test_restart_persistence.py::test_patient_persists_after_restart PASSED [ 25%]
tests/test_restart_persistence.py::test_doctor_persists_after_restart PASSED [ 50%]
tests/test_restart_persistence.py::test_appointment_persists_after_restart PASSED [ 75%]
tests/test_restart_persistence.py::test_medical_history_persists_after_restart PASSED [100%]

===== 4 passed in 0.19s =====

```

1.17 test_error_isolation.py

Purpose:

Tests that errors in one operation do not unnecessarily affect unrelated operations.

Result: PASS

```
tests/test_error_isolation.py::test_scheduling_error_does_not_crash_test PASSED [ 20%]
tests/test_error_isolation.py::test_double_booking_raises_exception PASSED [ 40%]
tests/test_error_isolation.py::test_missing_patient_raises_exception PASSED [ 60%]
tests/test_error_isolation.py::test_malformed_billing_input_raises_exception PASSED [ 80%]
tests/test_error_isolation.py::test_emergency_works_after_scheduling_failure PASSED [100%]

===== 5 passed in 0.12s =====
```

1.18 test_e2e.py

Purpose:

Tests the complete hospital workflow from registration and appointment scheduling to emergency processing and reporting.

Result: PASS

```
tests/test_e2e.py::test_full_hospital_day PASSED [ 50%]
tests/test_e2e.py::test_emergency_is_served_before_scheduled_appointment PASSED [100%]

===== 2 passed in 0.08s =====
```

2. Observations

1. Patient, doctor and appointment operations work correctly.
2. Emergency cases are processed according to severity and arrival order.
3. Appointment conflicts and invalid inputs are handled using custom exceptions.
4. Medical history supports retrieval and filtering.
5. Standard and insurance billing work correctly.
6. JSON persistence maintains data across application restarts.
7. Reports and CSV export provide the required hospital statistics.

3. Performance Insights

1. heapq provides efficient emergency priority handling.
2. Generators allow medical history to be processed lazily.
3. map() and filter() are used for report calculations.
4. JSON provides simple persistent storage for the project.

4. Challenges Faced

1. Implementing severity and arrival-order priority for emergencies.
2. Preventing appointment double booking.
3. Maintaining data after application restart.
4. Safely writing medical records without partial updates.
5. Implementing standard and insurance billing using the Strategy Pattern.

5. Conclusion

The unit and end-to-end tests verify the major functionalities of the Smart Hospital Management System and confirm that the components work correctly both independently and together.